

Vel Tech Group of Institutions

Name: UMA MAHESHWARI S

Email: vtu28408@veltech.edu.in

Roll no: vtu28408

Phone: null

Branch: VTU

Department: CSE with AIDS

Batch: 2028

Degree: B.Tech - Computer Science and Engineering with AIDS

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L2

VTU_DAA_Week 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 20

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
```

```
#define MAX 500 // Adjust based on expected problem constraints
```

```
int adj[MAX][MAX]; // Adjacency matrix for connections
int match[MAX];    // Stores which employee is assigned to a work
bool visited[MAX]; // Tracks visited works in each DFS run
```

```
// DFS to find an augmenting path for employee 'u'
```

```
bool can_match(int u, int total_works) {
    for (int v = 0; v < total_works; v++) {
        // If employee u can do work v and v hasn't been checked in this path
```

```

    if (adj[u][v] && !visited[v]) {
        visited[v] = true;

        // If work v is free OR the employee currently doing it can do something
    else
        if (match[v] < 0 || can_match(match[v], total_works)) {
            match[v] = u; // Assign/Reassign work v to employee u
            return true;
        }
    }
}
return false;
}

```

```

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

    int max_emp = 0, max_work = 0;
    int edges[N][2];

    // Initialize adjacency matrix
    memset(adj, 0, sizeof(adj));

    // Read connections and find the range of IDs
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &edges[i][0], &edges[i][1]);
        adj[edges[i][0]][edges[i][1]] = 1;

        if (edges[i][0] >= max_emp) max_emp = edges[i][0] + 1;
        if (edges[i][1] >= max_work) max_work = edges[i][1] + 1;
    }
}

```

```

int result = 0;
// Initialize all works as unassigned (-1)
memset(match, -1, sizeof(match));

```

```

// Try to find a work for each employee
for (int u = 0; u < max_emp; u++) {
    memset(visited, false, sizeof(visited));
    if (can_match(u, max_work)) {
        result++;
    }
}

```

```
}  
}  
printf("Maximum number of employees that could get works are: %d\n",  
result);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y , representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e , representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v , representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4

4

0 1

0 2

1 3

2 3

Output: 0 3

Answer

// You are using GCC

#include <stdio.h>

#include <stdbool.h>

#include <string.h>

#define MAX 101

int X, Y, E;

int adj[MAX][MAX];

int adj_size[MAX];

int matchX[MAX], matchY[MAX];

bool visitedX[MAX], visitedY[MAX];

// Standard DFS to find an augmenting path for matching

bool can_match(int u, bool visited[]) {

for (int i = 0; i < adj_size[u]; i++) {

int v = adj[u][i];

if (!visited[v]) {

visited[v] = true;

if (matchY[v] < 0 || can_match(matchY[v], visited)) {

matchY[v] = u;

matchX[u] = v;

return true;

}

}

}

return false;

}

// DFS to find reachable nodes via alternating paths (Unmatched edge X->Y,
Matched edge Y->X)

```

void find_reachable(int u) {
    visitedX[u] = true;
    for (int i = 0; i < adj_size[u]; i++) {
        int v = adj[u][i];
        // In alternating path: we move from X to Y using an edge NOT in matching
        if (matchX[u] != v && !visitedY[v]) {
            visitedY[v] = true;
            // Then move from Y back to X using the matched edge
            if (matchY[v] != -1 && !visitedX[matchY[v]]) {
                find_reachable(matchY[v]);
            }
        }
    }
}

```

```

int main() {
    if (scanf("%d %d", &X, &Y) != 2) return 0;
    if (scanf("%d", &E) != 1) return 0;

```

```

    for (int i = 1; i <= X; i++) adj_size[i] = 0;
    for (int i = 1; i <= X; i++) matchX[i] = -1;
    for (int i = 1; i <= Y; i++) matchY[i] = -1;

```

```

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][adj_size[u]++] = v;
    }

```

```

    // 1. Find Maximum Matching
    for (int i = 1; i <= X; i++) {
        bool visited[MAX] = {false};
        can_match(i, visited);
    }

```

```

    // 2. Identify reachable nodes starting from unmatched X vertices
    memset(visitedX, false, sizeof(visitedX));
    memset(visitedY, false, sizeof(visitedY));
    for (int i = 1; i <= X; i++) {
        if (matchX[i] == -1) {
            find_reachable(i);
        }
    }

```

```

    }
    // 3. Minimum Vertex Cover = (X \ ReachableX) ∪ (Y \ ReachableY)
    bool first = true;
    for (int i = 1; i <= X; i++) {
        if (!visitedX[i]) {
            if (!first) printf(" ");
            printf("X%d", i);
            first = false;
        }
    }
    for (int i = 1; i <= Y; i++) {
        if (visitedY[i]) {
            if (!first) printf(" ");
            printf("Y%d", i);
            first = false;
        }
    }
    printf("\n");

    return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <stdbool.h>
```

```
#define MAX_VERTICES 100
```

```
// Function to check if the graph is bipartite using BFS
```

```
bool isBipartite(int num_vertices, int adj[MAX_VERTICES][MAX_VERTICES]) {
```

```
    int color[MAX_VERTICES];
```

```
    for (int i = 0; i < num_vertices; i++) {
```

```
        color[i] = -1; // -1 means no color assigned
```

```
    }
```

```
    for (int i = 0; i < num_vertices; i++) {
```

```
        if (color[i] == -1) { // If the vertex is not colored, start BFS
```

```
            int queue[MAX_VERTICES];
```

```
            int head = 0, tail = 0;
```

```
            queue[tail++] = i;
```

```
            color[i] = 0; // Color the first vertex with 0
```

```
            while (head < tail) {
```

```
                int u = queue[head++];
```



```

    for (int v = 0; v < num_vertices; v++) {
        // If an edge exists and the neighbor has the same color
        if (adj[u][v] == 1) {
            if (color[v] == -1) {
                color[v] = 1 - color[u]; // Assign opposite color
                queue[tail++] = v;
            } else if (color[v] == color[u]) {
                return false; // Conflict found
            }
        }
    }
}

return true;
}

int main() {
    int num_vertices, num_edges;
    if (scanf("%d %d", &num_vertices, &num_edges) != 2) return 0;

    int adj[MAX_VERTICES][MAX_VERTICES] = {0};

    for (int i = 0; i < num_edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        if (u < num_vertices && v < num_vertices) {
            adj[u][v] = 1;
            adj[v][u] = 1;
        }
    }

    if (isBipartite(num_vertices, adj)) {
        printf("The graph is bipartite.");
    } else {
        printf("The graph is not bipartite.");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10